



DSA

Module 5

Arrays

Author
Srinivas Dande



My **JAVA**
LEARNING
CENTER



5. Arrays

- ♦ Array is a collection of elements which are of same data type.
- ♦ Array is also called as Homogeneous data structure.
- ♦ Elements of an array will be stored in contiguous memory locations.
- ♦ Arrays are objects in Java.
- ♦ Three tasks to remember while you are working with arrays:
 - Array Declaration
 - Array Construction
 - Array Initialization
- ♦ Arrays can be constructed with multiple dimensions i.e 1-D Array, 2-D Array etc.

5.1. Single Dimensional Arrays

- ♦ Single Dimensional array can be called as 1-D Array.

5.1.1. Array Declaration and Construction

- ♦ You can declare and construct an array in one statement also.

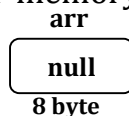
Syntax:

```
<dataType> <refVarName>[] = new <dataType>[<size>];  
<dataType> [] <refVarName> = new <dataType>[<size>];
```

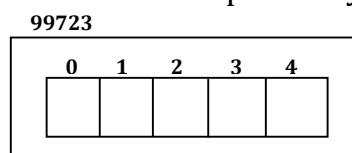
Ex:

```
int arr[] = new int[5];  
int [] arr = new int[5];
```

- 1) Allocates 8 bytes of memory for the reference variable and initializes with null value.

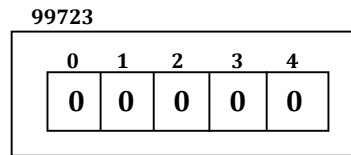


- 2) Memory blocks will be allocated as per array size.

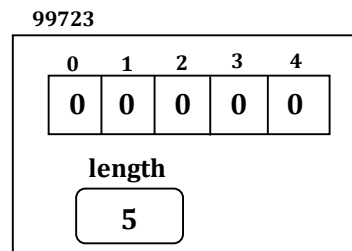




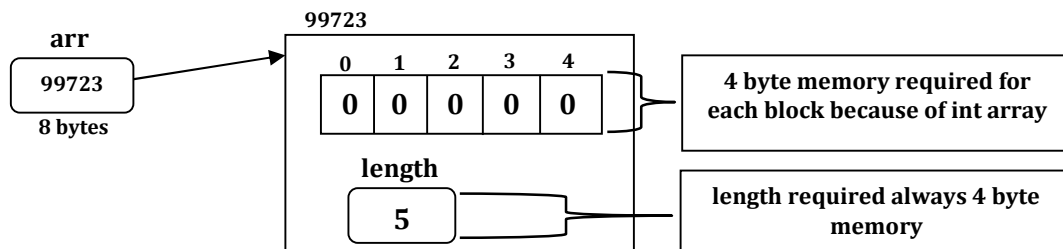
3) Memory blocks will be initialized with default value as per array data type.



4) Memory will be allocated for length variable and will be initialized with size of an array.

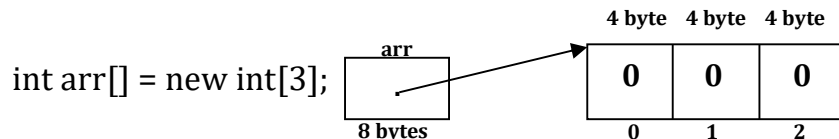


5) Array object address will be assigned to reference variable.



5.1.2. Array Initialization

- ◆ Once array is constructed then array elements will be initialized with default values as per array data type.
- ◆ You can initialize array elements with your own value using index representation.



- ◆ Here we are creating an array with size 3, so only three elements will be stored in this array.

`arr[0]` -> Represents the first element which contains value 0.

`arr[1]` -> Represents the second element which contains value 0.

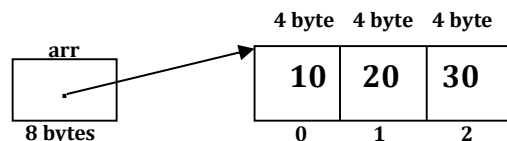
`arr[2]` -> Represents the third element which contains value 0.

- ◆ You can initialize the array elements as follows:

`arr[0] = 10;`

`arr[1] = 20;`

`arr[2] = 30;`



- ◆ Accessing the elements after initialization

`arr[0]` -> Represents the first element which contains value 10.

`arr[1]` -> Represents the second element which contains value 20.

`arr[2]` -> Represents the third element which contains value 30.



Lab1.java

```
package com.jlcindia.arrays;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

public class Lab1 {
    public static void main(String[] args) {

        // 1. Create Array
        int arr[] = new int[5];

        // 2.find the Length of Array
        int n = arr.length;
        System.out.println(n);

        // 3.Access Elements of Array
        for (int i = 0; i < n; i++)
            System.out.println(arr[i]);

        // 4. Initialize the Array
        arr[0] = 11;
        arr[2] = 33;
        arr[4] = 55;
        System.out.println("-----");

        // Access Elements of Array
        for (int i = 0; i < n; i++)
            System.out.println(arr[i]);
    }
}
```



5.1.3 Array Declaration, Construction and Initialization

- ♦ You can declare, construct and initialize an array in one statement also.

Syntax:

```
<dataType> <refVarName>[] = {<val1>,<val2>,<val3>,...,<valN>};
```

Ex:

```
int arr[]={10,20,30};  
String names[]{"Hello","Srinivas","Dande"};
```

Lab2.java

```
package com.jlcindia.arrays;  
  
/*  
 * @Author : Srinivas Dande  
 * @Company: Java Learning Center  
 */  
  
public class Lab2{  
    public static void main(String[] args) {  
  
        int arr[] = {10,20,30,40,50};  
  
        int n= arr.length;  
  
        // Access Elements of Array  
        for (int i = 0; i < n; i++)  
            System.out.println(arr[i]);  
    }  
}
```



5.1.4. Why and Why Not Arrays?

♦ Limitations with array:

- ✓ Array is a homogeneous data structure i.e only one type of elements can be stored in the Array.
- ✓ Array is static in nature i.e size of an array cannot be modified.
- ✓ Insert and delete operations require more shifting of elements.
- ✓ You need to write the Code for doing any Operations on Arrays. No Built-In Functionality provided in Arrays.

♦ Advantages with array:

- ✓ Arrays are faster in Accessing the Elements.
- ✓ Arrays are Cache-Friendly



5.1.5 Dynamic Size Array - ArrayList

- ♦ ArrayList is implemented as a resizable array.
- ♦ ArrayList elements will be stored internally using indexing notation.
- ♦ This is one of the most widely used concrete class.
- ♦ It is fast to access the elements, but slow to insert and delete the elements.
- ♦ Size of ArrayList will be growable dynamically.
- ♦ Default Capacity is 10. i.e When You create ArrayList, It allocates the Space for 10 Elements in advance.
- ♦ When you add add elements then size will be increased.
- ♦ Following the Formula for Capacity Calculation.

In Java 6

$$\text{newCapacity} = (\text{oldCapacity} * 3) / 2 + 1;$$

In Java 7

$$\text{newCapacity} = \text{oldCapacity} + (\text{oldCapacity} >> 1)$$

- ♦ ArraList comes with many built-methods as follows.
 - `int size()`
 - `boolean isEmpty()`
 - `Object get(int idx)`
 - `boolean add(Object obj)`
 - `void add(int idx, Object obj)`
 - `boolean remove(Object obj)`
 - `Object remove(int idx)`
 - `Object set(int idx, Object obj)`
 - `boolean contains(Object obj)`



Lab3.java

```
package com.jlcindia.arrays;

/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

import java.util.ArrayList;
import java.util.List;

public class Lab3{
    public static void main(String[] args) {

        List<Integer> mylist = new ArrayList<>();

        mylist.add(10);
        mylist.add(20);
        mylist.add(30);
        mylist.add(40);

        System.out.println(mylist);
        mylist.add(50);
        System.out.println(mylist);
        mylist.remove(Integer.valueOf(40));
        System.out.println(mylist);
        //ArrayList has many Built-In methods to Use
    }
}
```



5.2. Operations on Arrays

Lab4.java

```
package com.jlcindia.arrays;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
//Search for given Element in the Unsorted Array

public class Lab4 {
    static int linearSearch(int arr[], int element) {
        int n = arr.length;

        for (int i = 0; i < n; i++) {
            if (arr[i] == element) {
                return i;
            }
        }
        return -1;
    }

    public static void main(String[] args) {
        int arr[] = { 25, 10, 5, 15, 30, 20, 50 };
        int element = 25;

        int index = linearSearch(arr, element);

        System.out.println(index);
        if (index == -1)
            System.out.println("Element Not Found");
        else
            System.out.println("Element Found at Index : " + index);
    }
}

// Time Complexity - - L.S O(n)
// Aux Space - O(1)
```



Lab5.java

```
package com.jlcindia.arrays;

/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */

import java.util.Arrays;

//Array Operations
//2. Insert the element at given position
//When There is No Space to Insert, then This Algorithm will not Work

public class Lab5 {
    public static void insert(int arr[], int element, int position) {
        int n = arr.length;
        int myindex = position - 1;

        for (int i = n - 1; i > myindex; i--) {
            arr[i] = arr[i - 1];
        }

        arr[myindex] = element;
    }

    public static void main(String[] args) {

        int arr[] = new int[8];
        arr[0] = 10;
        arr[1] = 20;
        arr[2] = 30;
        arr[3] = 40;
        arr[4] = 50;

        System.out.println(Arrays.toString(arr));
    }
}
```



```
        insert(arr, 99, 3);
        System.out.println(Arrays.toString(arr));

        insert(arr, 60, 7);
        System.out.println(Arrays.toString(arr));

        insert(arr, 88, 3);
        System.out.println(Arrays.toString(arr));

        insert(arr, 77, 3);
        System.out.println(Arrays.toString(arr));
    }
}
// Time Complexity - - O(n)
// Aux Space - O(1)
```

Lab6.java

```
package com.jlcindia.arrays;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
import java.util.Arrays;
//Array Operations
//3. Delete the element at given position
public class Lab6 {
    public static void deleteAt(int arr[], int position) {

        int n = arr.length;
        int myindex= position-1;

        for(int i=myindex;i<=n-2;i++) {
            arr[i] = arr[i+1];
        }
        arr[n-1] = 0;
    }
}
```



```
public static void main(String[] args) {  
  
    int arr[] = new int[6];  
    arr[0] = 10;  
    arr[1] = 20;  
    arr[2] = 30;  
    arr[3] = 40;  
    arr[4] = 50;  
    arr[5] = 60;  
  
    System.out.println(Arrays.toString(arr));  
  
    deleteAt(arr,3);  
    System.out.println(Arrays.toString(arr));  
  
    deleteAt(arr,3);  
    System.out.println(Arrays.toString(arr));  
  
    deleteAt(arr,3);  
    System.out.println(Arrays.toString(arr));  
}  
}
```

// Time Complexity - - $O(n)$
// Aux Space - $O(1)$



Lab7.java

```
package com.jlcindia.arrays;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
import java.util.Arrays;

//Array Operations
//3. Delete the given element
public class Lab7 {
    public static int linearSearch(int arr[], int element) {
        for (int i = 0; i < arr.length; i++) {
            if (arr[i] == element)
                return i;
        }
        return -1;
    }
    public static void deleteAt(int arr[], int position) {

        int n = arr.length;
        int myindex = position - 1;

        for (int i = myindex; i <= n - 2; i++) {
            arr[i] = arr[i + 1];
        }

        arr[n - 1] = 0;
    }
    public static void deleteElement(int arr[], int element) {

        int myindex = linearSearch(arr, element);
        if (myindex != -1)
            deleteAt(arr, myindex + 1);

    }
}
```



```
public static void main(String[] args) {  
  
    int arr[] = new int[6];  
    arr[0] = 10;  
    arr[1] = 20;  
    arr[2] = 30;  
    arr[3] = 40;  
    arr[4] = 50;  
    arr[5] = 60;  
  
    System.out.println(Arrays.toString(arr));  
  
    deleteElement(arr, 30);  
    System.out.println(Arrays.toString(arr));  
  
    deleteElement(arr, 90);  
    System.out.println(Arrays.toString(arr));  
  
    deleteElement(arr, 40);  
    System.out.println(Arrays.toString(arr));  
  
    }  
}
```

// Time Complexity - - $O(n)$
// Aux Space - $O(1)$



Lab8.java

```
package com.jlcindia.arrays;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
import java.util.Arrays;

//Array Operations
//4. Update the element at given position
public class Lab8 {
    public static void update(int arr[], int newElement, int position) {
        arr[position - 1] = newElement;
    }

    public static void main(String[] args) {

        int arr[] = new int[6];
        arr[0] = 10;
        arr[1] = 20;
        arr[2] = 30;
        arr[3] = 40;
        arr[4] = 50;
        arr[5] = 60;

        System.out.println(Arrays.toString(arr));

        update(arr, 99, 3);
        System.out.println(Arrays.toString(arr));

        update(arr, 88, 4);
        System.out.println(Arrays.toString(arr));
    }
}
// Time Complexity - O(1)
//Aux Space - O(1)
```



Lab9.java

```
package com.jlcindia.arrays;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
import java.util.Arrays;

//Array Operations
//4. Update the element at given position
public class Lab9 {
    public static int linearSearch(int arr[], int element) {
        for (int i = 0; i < arr.length; i++) {
            if (arr[i] == element)
                return i;
        }
        return -1;
    }

    public static void updateElement(int arr[], int oldElement, int newElement) {
        int myindex = linearSearch(arr, oldElement);
        if (myindex != -1) {
            arr[myindex] = newElement;
        }
    }

    public static void main(String[] args) {

        int arr[] = new int[6];
        arr[0] = 10;
        arr[1] = 20;
        arr[2] = 30;
        arr[3] = 40;
        arr[4] = 50;
        arr[5] = 60;
    }
}
```



```
        System.out.println(Arrays.toString(arr));

        updateElement(arr, 30, 99);
        System.out.println(Arrays.toString(arr));

        updateElement(arr, 40, 88);
        System.out.println(Arrays.toString(arr));

    }
}
```

// Time Complexity - O(n)
// Aux Space - O(1)

Lab10.java

```
package com.jlcindia.arrays;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
//Array Operations
//5. Access the Elements of Array
public class Lab10 {
    public static void main(String[] args) {

        int arr[] = {10,20,30,40,50};

        System.out.println(arr[0]);
        System.out.println(arr[1]);
        System.out.println(arr[2]);
        System.out.println(arr[3]);
        System.out.println(arr[4]);

        for(int x:arr)
            System.out.println(x);

    }
}
```



5.3. Time Complexity of Array Operations

Operation	Time Complexity
Access One Element	$O(1)$
Access All Elements	$O(n)$
Update One Element	$O(1)$
Linear Search	$O(n)$
Binary Search	$O(\log n)$
Insert Element at nth postion	$O(n)$
Insert Element at First postion	$O(n)$
Insert Element at Last postion	$O(1)$
Delete Element at nth postion	$O(n)$
Delete Element at First postion	$O(n)$
Delete Element at Last postion	$O(1)$



Lab11.java

```
package com.jlcindia.arrays;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
//Array Implementation
import java.util.Arrays;
class MyArray {
    int size;
    int capacity;
    int[] arr;

    MyArray() {
        this.size = 0;
        this.capacity = 10;
        this.arr = new int[capacity];
    }

    MyArray(int capacity) {
        this.size = 0;
        this.capacity = capacity;
        this.arr = new int[capacity];
    }

    public int size() {
        return size;
    }

    public int capacity() {
        return capacity;
    }

    public boolean isEmpty() {
        return (size == 0);
    }
}
```



```
public void add(int element) {
    if (size == capacity) {
        resizeArray();
    }

    arr[size] = element;
    size++;
}

public void add(int position, int element) {
    if (size == capacity) {
        resizeArray();
    }

    int myindex = position - 1;

    for (int i = capacity - 1; i > myindex; i--) {
        arr[i] = arr[i - 1];
    }

    arr[myindex] = element;
    size++;
}

public void removeAt(int index) {

    for (int i = index; i <= size; i++) {
        arr[i] = arr[i + 1];
    }

    arr[size - 1] = 0;
    size--;
}
```



```
public void removeElement(int element) {

    int myindex = -1;
    for (int i = 0; i < arr.length; i++) {
        if (arr[i] == element)
            myindex = i;
    }

    if (myindex != -1) {
        removeAt(myindex);
    }

}

public boolean contains(int element) {
    for (int i = 0; i < arr.length; i++) {
        if (arr[i] == element)
            return true;
    }
    return false;
}

public void update(int index, int newElement) {
    arr[index] = newElement;
}

public void clear() {

    this.size = 0;

    // Down-size the Array
    this.capacity = 10;
    int temp[] = new int[this.capacity];
    this.arr = temp;

}
```



```
public String toString() {
    String str = "[";
    for (int i = 0; i < size; i++) {
        str = str + "" + arr[i] + ",";
    }

    if (size != 0)
        str = str.substring(0, str.length() - 1);

    str = str + "]";
    return str;
}

private void resizeArray() {
    // Resize the Array
    int newCapacity = capacity + (capacity >> 1);
    int temp[] = new int[newCapacity];
    System.arraycopy(arr, 0, temp, 0, capacity);

    this.arr = temp;
    this.capacity = newCapacity;
}

}

public class Lab11 {
    public static void main(String[] args) {
        MyArray array = new MyArray(5);
        System.out.println(array);
        System.out.println(array.size());
        System.out.println(array.capacity());
        System.out.println(array.isEmpty());

        array.add(10);
        array.add(20);
        array.add(30);
        array.add(-25);
    }
}
```



```
System.out.println(array);
System.out.println(array.size());
System.out.println(array.capacity());
System.out.println(array.isEmpty());

array.add(3, 99);
array.add(3, 88); // **
array.add(3, 77);

System.out.println(array);
System.out.println(array.size());
System.out.println(array.capacity());

array.add(123); // *
array.add(321);
System.out.println(array);
System.out.println(array.size());
System.out.println(array.capacity());

array.add(555);
array.add(666); // **
System.out.println(array);
System.out.println(array.size());
System.out.println(array.capacity());

array.removeAt(0);
System.out.println(array);
array.removeAt(1);
System.out.println(array);
System.out.println(array.size());
System.out.println(array.capacity());

System.out.println(array.contains(99));
System.out.println(array.contains(999));
```



```
array.removeElement(-25);
array.removeElement(20);
array.removeElement(30);

System.out.println(array);
System.out.println(array.size());
System.out.println(array.capacity());

array.update(0, 888);
array.update(1, 999);
System.out.println(array);
System.out.println(Arrays.toString(array.arr));

array.clear();
System.out.println(array);
System.out.println(array.size());
System.out.println(array.capacity());
System.out.println(Arrays.toString(array.arr));

    }
}
```



5.4. Problems on Arrays

Problem1A.java

```
package com.jlcindia.arrays;
import java.util.Arrays;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
//Problem1A- Reverse the Array
public class Problem1A {

    public static void reverse(int arr[]) {

        int n = arr.length;
        int revArr[] = new int[n];

        for (int i = n - 1, j = 0; j <= n - 1; i--, j++)
            revArr[i] = arr[j];

        for (int i = 0; i <= n - 1; i++)
            arr[i] = revArr[i];
    }

    public static void main(String[] args) {

        int arr[] = { 10, 20, 30, 40, 50, 60 };
        System.out.println(Arrays.toString(arr));
        reverse(arr);
        System.out.println(Arrays.toString(arr));
    }
}

// Time Complexity - O(n)
//Aux Space Complexity - O(n)
```



Problem1B.java

```
package com.jlcindia.arrays;

import java.util.Arrays;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
//Problem1B- Reverse the Array
//Input Array should not be modified
public class Problem1B {

    public static int [] reverse(int arr[]) {

        int n = arr.length;
        int revArr[] = new int[n];

        for (int i = n - 1, j = 0; j <= n - 1; i--, j++)
            revArr[i] = arr[j];

        return revArr;
    }

    public static void main(String[] args) {

        int arr[] = { 10, 20, 30, 40, 50, 60 };
        System.out.println(Arrays.toString(arr));
        int revArr[] = reverse(arr);
        System.out.println(Arrays.toString(arr));
        System.out.println(Arrays.toString(revArr));
    }
}

// Time Complexity - O(n)
//Aux Space Complexity - O(n)
```



Problem1C.java

```
package com.jlcindia.arrays;

import java.util.Arrays;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
//Problem1C- Reverse the Array
//Problem should be solved with Constant Space
// Two Pointers Technique
public class Problem1C {
    public static void reverse(int arr[]) {

        int start = 0;
        int end = arr.length - 1;

        while (start < end) {
            int temp = arr[start];
            arr[start] = arr[end];
            arr[end] = temp;

            start++;
            end--;
        }
    }

    public static void main(String[] args) {

        int arr[] = { 10, 20, 30, 40, 50, 60, 70 };
        System.out.println(Arrays.toString(arr));
        reverse(arr);
        System.out.println(Arrays.toString(arr));
    }
}
// Time Complexity - O(n/2)
//Aux Space Complexity - O(1)
```



Problem2.java

```
package com.jlcindia.arrays;

import java.util.Arrays;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
//Problem 2: Index of Max element in the Array
public class Lab2 {
    public static int maxIndex(int arr[]) {

        int n = arr.length;
        int maxIndex = 0;

        for (int i = 1; i < n; i++) {
            if (arr[i] > arr[maxIndex]) {
                maxIndex = i;
            }
        }
        return maxIndex;
    }

    public static void main(String[] args) {

        int arr[] = { 20, 5, 25, 9, 30, 55, 22 };
        System.out.println(Arrays.toString(arr));

        int maxIndex = maxIndex(arr);
        System.out.println(maxIndex);
    }
}
// Time Complexity - - O(n)
// Aux Space - O(1)
```



Problem3.java

```
package com.jlcindia.arrays;

import java.util.Arrays;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
//Problem3- Find Max Element
public class Problem3 {

    public static int maxElement(int arr[]) {

        int max = arr[0];
        for (int i = 1; i <= arr.length - 1; i++) {
            if (arr[i] > max) {
                max = arr[i];
            }
        }
        return max;
    }

    public static void main(String[] args) {

        int arr[] = { 50, 30, 95, 60, 10, 40, 90, 70 };
        System.out.println(Arrays.toString(arr));

        int x = maxElement(arr);
        System.out.println(x);
    }
}

// Time Complexity - O(n)
//Aux Space Complexity - O(1)
```



Problem4.java

```
package com.jlcindia.arrays;

import java.util.Arrays;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
//Problem4- Find Index of Min Element
public class Problem4 {

    public static int minIndex(int arr[]) {

        int minIndex = 0;
        for (int i = 1; i <= arr.length - 1; i++) {
            if (arr[i] < arr[minIndex]) {
                minIndex = i;
            }
        }

        return minIndex;
    }

    public static void main(String[] args) {

        int arr[] = { 50, 30, 20, 60, 10, 40, 90, 70 };
        System.out.println(Arrays.toString(arr));

        int x = minIndex(arr);
        System.out.println(x);
    }
}

// Time Complexity - O(n)
//Aux Space Complexity - O(1)
```



Problem5.java

```
package com.jlcindia.arrays;

import java.util.Arrays;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
//Problem3- Find Min Element
public class Problem5 {

    public static int minElement(int arr[]) {

        int min = arr[0];
        for (int i = 1; i <= arr.length - 1; i++) {
            if (arr[i] < min) {
                min = arr[i];
            }
        }
        return min;
    }

    public static void main(String[] args) {

        int arr[] = { 50, 30, 95, 60, 10, 40, 90, 70, 5 };
        System.out.println(Arrays.toString(arr));

        int x = minElement(arr);
        System.out.println(x);
    }
}

// Time Complexity - O(n)
//Aux Space Complexity - O(1)
```



Problem6.java

```
package com.jlcindia.arrays;

import java.util.Arrays;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
//Problem6- Print Extreme Elements of Array
//Two Pointers Technique
public class Problem6 {
    public static void extremePrint(int arr[]) {

        int start = 0;
        int end = arr.length - 1;

        while (start <= end) {

            if (start == end) {
                System.out.println(arr[start]);
            } else {
                System.out.println(arr[start]);
                System.out.println(arr[end]);
            }
            start++;
            end--;
        }
    }

    public static void main(String[] args) {
        int arr[] = { 10, 20, 30, 40, 50, 60, 70 };
        System.out.println(Arrays.toString(arr));
        extremePrint(arr);
    }
}

// Time Complexity - O(n)
//Aux Space Complexity - O(1)
```



Problem7A.java

```
package com.jlcindia.arrays;

import java.util.Arrays;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
//Problem7A- Contans Duplicates (LC - 217)
//Brute Force Technique
public class Problem7A {

    public static boolean containsDuplicate(int[] arr) {

        int n = arr.length;
        for(int i=0;i<=n-1;i++) {
            for(int j=i+1;j<=n-1;j++) {
                if(arr[i]==arr[j]) {
                    return true;
                }
            }
        }
        return false;
    }

    public static void main(String[] args) {
        int arr[] = { 10, 20, 30, 40, 50, 60, 70,50 };
        System.out.println(Arrays.toString(arr));

        boolean x = containsDuplicate(arr);
        System.out.println(x);
    }
}

// Time Complexity -  $O(n^2)$ 
//Aux Space Complexity -  $O(1)$ 
```



Problem7B.java

```
package com.jlcindia.arrays;

import java.util.Arrays;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
//Problem7B- Contans Duplicates (LC - 217)
//using Sorting
public class Problem7B {
    public static boolean containsDuplicate(int[] arr) {

        Arrays.sort(arr);
        int n = arr.length;

        for (int i = 0; i <= n - 2; i++) {
            if (arr[i] == arr[i + 1]) {
                return true;
            }
        }
        return false;
    }

    public static void main(String[] args) {

        int arr[] = { 10, 20, 30, 40, 60, 50 };
        System.out.println(Arrays.toString(arr));

        boolean x = containsDuplicate(arr);
        System.out.println(x);
    }
}
// Time Complexity - O(n log n)
//Aux Space Complexity - O(n)
```



Problem7C.java

```
package com.jlccindia.arrays;

import java.util.*;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
//Problem7C- Contans Duplicates (LC - 217)
//using HashSet
public class Problem7C {
    public static boolean containsDuplicate(int[] arr) {

        HashSet<Integer> myset = new HashSet<>();
        int n = arr.length;

        for (int i = 0; i <= n - 1; i++) {
            if (myset.contains(arr[i])) {
                return true;
            }
            myset.add(arr[i]);
        }
        return false;
    }
    public static void main(String[] args) {

        int arr[] = { 10, 20, 30, 40, 60, 50,30 };
        System.out.println(Arrays.toString(arr));

        boolean x = containsDuplicate(arr);
        System.out.println(x);
    }
}
// Time Complexity - O(n )
//Aux Space Complexity - O(n)
```



Problem8A.java

```
package com.jlcindia.arrays;

import java.util.Arrays;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
//Problem8A- Find the Duplicate Number( LC - 287)
//Brute Force Technique
public class Problem8A {
    public static int findDuplicate(int[] arr) {

        int n = arr.length;
        for(int i=0;i<=n-1;i++) {
            for(int j=i+1;j<=n-1;j++) {
                if(arr[i]==arr[j]) {
                    return arr[i];
                }
            }
        }
        return -1;
    }
    public static void main(String[] args) {

        int arr[] = { 10, 20, 30, 40, 50, 60, 70,90 };
        System.out.println(Arrays.toString(arr));

        int x = findDuplicate(arr);
        System.out.println(x);
    }
}
// Time Complexity -  $O(n^2)$ 
//Aux Space Complexity -  $O(1)$ 
```



Problem8B.java

```
package com.jlcindia.arrays;

import java.util.Arrays;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
//Problem7B- Contans Duplicates(LC - 217)
//using Sorting
public class Problem8B {
    public static int findDuplicate(int[] arr) {

        Arrays.sort(arr);
        int n = arr.length;

        for (int i = 0; i <= n - 2; i++) {
            if (arr[i] == arr[i + 1]) {
                return arr[i];
            }
        }
        return -1;
    }

    public static void main(String[] args) {

        int arr[] = { 10, 20, 30, 40, 60, 50, 60 };
        System.out.println(Arrays.toString(arr));

        int x = findDuplicate(arr);
        System.out.println(x);
    }
}
// Time Complexity - O(n log n)
//Aux Space Complexity - O(n)
```



Problem8C.java

```
package com.jlccindia.arrays;

import java.util.*;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
//Problem7C- Contans Duplicates (LC - 217)
//using HashSet
public class Problem8C {

    public static int findDuplicate(int[] arr) {

        HashSet<Integer> myset = new HashSet<>();
        int n = arr.length;

        for (int i = 0; i <= n - 1; i++) {
            if (myset.contains(arr[i])) {
                return arr[i];
            }
            myset.add(arr[i]);
        }
        return -1;
    }

    public static void main(String[] args) {
        int arr[] = { 10, 20, 30, 40, 60, 50, 90 };
        System.out.println(Arrays.toString(arr));

        int x = findDuplicate(arr);
        System.out.println(x);
    }
}

// Time Complexity - O(n )
//Aux Space Complexity - O(n)
```



Problem9.java

```
package com.jlcindia.arrays;

import java.util.Arrays;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
//Problem9- Find the Unique Element (Single Number)
//LeetCode - 136
public class Problem9 {
    public static int singleNumber(int[] nums) {

        int result=0;
        for(int i=0;i<nums.length;i++) {
            result = result ^ nums[i];
        }
        return result;
    }

    public static void main(String[] args) {

        int arr[] = { 5,1,2,1,2,6,6};
        System.out.println(Arrays.toString(arr));

        int x = singleNumber(arr);
        System.out.println(x);
    }
}

// Time Complexity - O(n )
//Aux Space Complexity - O(1)
```



Problem10A.java

```
package com.jlcindia.arrays;
import java.util.*;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
//Problem10- Missing Number ( LC - 268 )
public class Problem10A {
    public static int missingNumber(int[] arr) {

        HashSet<Integer> myset = new HashSet<>();

        for(int x: arr) {
            myset.add(x);
        }

        for(int i=0;i<arr.length+1;i++) {
            if(!myset.contains(i)) {
                return i;
            }
        }
        return -1;
    }
    public static void main(String[] args) {
        int arr[] = { 3, 0, 2, 1, 4, 6 };
        System.out.println(Arrays.toString(arr));

        int x = missingNumber(arr);
        System.out.println(x);
    }
}
// Time Complexity - O(n )
//Aux Space Complexity - O(n)
```



Problem10B.java

```
package com.jlcindia.arrays;
import java.util.*;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
//Problem10- Missing Number (LC - 268)
public class Problem10B {
    public static int missingNumber(int[] arr) {

        int n = arr.length;
        int actualsum = 0;

        for (int x : arr) {
            actualsum = actualsum + x;
        }

        int expSum = n * (n + 1) / 2;

        return expSum - actualsum;
    }

    public static void main(String[] args) {
        int arr[] = { 3, 0, 1 };
        System.out.println(Arrays.toString(arr));

        int x = missingNumber(arr);
        System.out.println(x);
    }
}

// Time Complexity - O(n )
//Aux Space Complexity - O(1)
```



Problem11.java

```
package com.jlcindia.arrays;
import java.util.*;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
//Problem11- Kth Largest Element in an Array (LC - 215)
public class Problem11 {
    public static int findKthLargest(int[] arr, int k) {

        PriorityQueue<Integer> pqque = new PriorityQueue<>();

        for (int x : arr) {
            pqque.add(x);

            if (pqque.size() > k) {
                pqque.poll();
            }
        }
        return pqque.peek();
    }
    public static void main(String[] args) {

        int arr[] = { 12, 30, 28, 10, 15, 5, 27 };
        int k = 3;
        System.out.println(Arrays.toString(arr));

        int x = findKthLargest(arr, k);
        System.out.println(x);

    }
}
// Time Complexity - O(n log k)
//Aux Space Complexity - O(k)
```



Problem12.java

```
package com.jlcindia.arrays;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
//Problem12- Kth Smallest Element in an Array
public class Problem12 {

    public static int findKthSmallest(int[] arr, int k) {

        PriorityQueue<Integer> pqque = new
            PriorityQueue<>(Comparator.reverseOrder());

        for (int x : arr) {
            pqque.add(x);

            if (pqque.size() > k) {
                pqque.poll();
            }
        }
        return pqque.peek();
    }
    public static void main(String[] args) {
        int arr[] = { 12, 30, 28, 10, 15, 5, 27 };
        int k = 3;
        System.out.println(Arrays.toString(arr));

        int x = findKthSmallest(arr, k);
        System.out.println(x);
    }
}

// Time Complexity - O(n log k )
//Aux Space Complexity - O(k)
```



Problem13.java

```
package com.jlcindia.arrays;
import java.util.Arrays;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
//Problem 13: Array is Sorted in ASC or Not
public class Problem13 {

    public static boolean isSortedASC(int[] arr) {
        int n = arr.length;

        for (int i = 1; i < n; i++) {
            if (arr[i - 1] > arr[i])
                return false;
        }
        return true;
    }

    public static void main(String[] args) {

        int arr[] = { 10,20,30,90,50,60 };
        System.out.println(Arrays.toString(arr));

        boolean x = isSortedASC(arr);
        System.out.println(x);
    }
}

// Time Complexity - - O(n)
// Aux Space - O(1)
```



Problem14.java

```
package com.jlcindia.arrays;
import java.util.Arrays;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
//Problem 14: Array is Sorted in DESC or Not
public class Problem4 {

    public static boolean isSortedDESC(int[] arr) {
        int n = arr.length;

        for (int i = 1; i < n; i++) {
            if (arr[i - 1] < arr[i])
                return false;
        }
        return true;
    }

    public static void main(String[] args) {

        int arr[] = { 60,50,40,30,20,10 };
        System.out.println(Arrays.toString(arr));

        boolean x = isSortedDESC(arr);
        System.out.println(x);
    }
}

// Time Complexity - - O(n)
// Aux Space - O(1)
```



Problem15.java

```
package com.jlcindia.arrays;
import java.util.Arrays;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
//Problem 15: Move All Zeors to End
//LeetCode: 283
public class Problem5 {
    public static void moveZeroes(int[] arr) {
        int n = arr.length;
        int k = 0;

        for (int i = 0; i < n; i++) {
            if (arr[i] != 0) {
                int x = arr[i];
                arr[i] = arr[k];
                arr[k] = x;

                k++;
            }
        }
    }

    public static void main(String[] args) {
        int arr[] = { 0, 20, 0, 30, 10, 0, 30, 60 };
        //int arr[] = { 0, -20, 0, 30, -10, 0, -30, 60 };
        System.out.println(Arrays.toString(arr));

        moveZeroes(arr);
        System.out.println(Arrays.toString(arr));
    }
}
// Time Complexity - - O(n)
// Aux Space - O(1)
```

Problem16.java

```
package com.jlcindia.arrays;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
//Problem 16: Remove Element from Array
//LeetCode: 27
public class Problem16 {
    public static int removeElement(int[] arr, int element) {
        int n = arr.length;
        int k = 0;

        for (int i = 0; i < n; i++) {
            if (arr[i] != element) {
                arr[k] = arr[i];
                k++;
            }
        }
        for(int i=k;i<n;i++) {
            arr[i] = 0;
        }
        return k;
    }
    public static void main(String[] args) {
        int arr[] = { 30, 20, 50, 30, 10, 40, 30, 60 };
        int element=30;
        System.out.println(Arrays.toString(arr));

        int count = removeElement(arr, element);
        System.out.println("Unique Count : " + count);
        System.out.println(Arrays.toString(arr));
    }
}
// Time Complexity - - O(n)
// Aux Space - O(1)
```



Problem17.java

```
package com.jlccindia.arrays;
import java.util.Arrays;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
//Problem 17: Remove the Duplicates from Sorted Array
//LeetCode - 26
public class Lab17 {
    public static int removeDuplicates(int arr[]) {
        int n=arr.length;
        int k = 1;

        for(int i=1;i<n;i++) {
            if(arr[i]!=arr[k-1]) {
                arr[k] =arr[i];
                k++;
            }
        }
        for(int i=k;i<n;i++) {
            arr[i] = 0;
        }
        return k;
    }
    public static void main(String[] args) {
        int arr[] = { 10,20,20,20,20,30,30,40,50,50,60,70};
        System.out.println(Arrays.toString(arr));

        int count= removeDuplicates(arr);
        System.out.println("Unique Count :"+ count);
        System.out.println(Arrays.toString(arr));
    }
}
// Time Complexity - - O(n)
// Aux Space - O(1)
```



Problem18.java

```
package com.jlcindia.arrays;
import java.util.Arrays;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
// Problem 18: Rotate the Elements Left side 1 time.
public class Problem18 {
    public static void rotateLeftOne(int[] arr) {

        int n = arr.length;
        int x = arr[0];

        for (int i = 1; i < n; i++) {
            arr[i - 1] = arr[i];
        }
        arr[n - 1] = x;
    }
    public static void main(String[] args) {

        int arr[] = { 1, 2, 3, 4, 5, 6 };
        System.out.println(Arrays.toString(arr));

        rotateLeftOne(arr);
        System.out.println(Arrays.toString(arr));

        rotateLeftOne(arr);
        System.out.println(Arrays.toString(arr));
    }
}

// Time Complexity - - O(n)
// Aux Space - O(1)
```



Problem19.java

```
package com.jlcindia.arrays;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
// Problem 19: Rotate the Elements Right side 1 time.
public class Problem19 {
    public static void rotateRightOne(int[] arr) {

        int n = arr.length;
        int x = arr[n-1];

        for (int i = n-1; i>=1; i--) {
            arr[i] = arr[i-1];
        }
        arr[0] = x;
    }

    public static void main(String[] args) {

        int arr[] = { 1, 2, 3, 4, 5, 6 };
        System.out.println(Arrays.toString(arr));

        rotateRightOne(arr);
        System.out.println(Arrays.toString(arr));

        rotateRightOne(arr);
        System.out.println(Arrays.toString(arr));
    }
}

// Time Complexity - - O(n)
// Aux Space - O(1)
```



Problem20A.java

```
package com.jlccindia.arrays;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
//Problem 20A: Rotate the Elements Left side K times.
public class Problem20A {
    public static void rotateLeftOne(int[] arr) {
        int n = arr.length;
        int x = arr[0];

        for (int i = 1; i < n; i++) {
            arr[i - 1] = arr[i];
        }
        arr[n - 1] = x;
    }
    public static void rotateLeft(int[] arr, int k) {

        for (int i = 1; i <= k; i++) {
            rotateLeftOne(arr);
        }
    }
    public static void main(String[] args) {
        int arr[] = { 1, 2, 3, 4, 5, 6 };
        System.out.println(Arrays.toString(arr));

        rotateLeft(arr, 6);
        System.out.println(Arrays.toString(arr));
    }
}

// Time Complexity - - O(kn)
// Aux Space - O(1)
```



Problem20B.java

```
package com.jlcindia.arrays;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
// Problem 20B: Rotate the Elements Left side K times.
public class Problem20B {
    public static void reverse(int arr[],int start,int end) {
        while(start<end) {
            int temp = arr[start];
            arr[start] = arr[end];
            arr[end] = temp;

            start++;
            end--;
        }
    }
    public static void rotateLeft(int[] arr, int k) { // k=3
        int n=arr.length;
        k=k%n;

        reverse(arr,0,k-1);
        reverse(arr,k,n-1);
        reverse(arr,0,n-1);
    }
    public static void main(String[] args) {
        int arr[] = { 1, 2, 3, 4, 5, 6 };
        System.out.println(Arrays.toString(arr));

        rotateLeft(arr,9);
        System.out.println(Arrays.toString(arr));
    }
}
// Time Complexity - - O(n)
// Aux Space - O(1)
```

Problem21A.java

```
package com.jlcindia.arrays;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
// Problem 21A: Rotate the Elements Right side K times.
public class Problem21A {
    public static void rotateRightOne(int[] arr) {
        int n = arr.length;
        int x = arr[n - 1];

        for (int i = n - 1; i >= 1; i--) {
            arr[i] = arr[i - 1];
        }
        arr[0] = x;
    }
    public static void rotateRight(int[] arr, int k) {
        for (int i = 1; i <= k; i++) {
            rotateRightOne(arr);
        }
    }
    public static void main(String[] args) {
        int arr[] = { 1, 2, 3, 4, 5, 6, 7 };
        System.out.println(Arrays.toString(arr));

        rotateRight(arr, 3);
        System.out.println(Arrays.toString(arr));
    }
}

// Time Complexity - - O(kn)
// Aux Space - O(1)
```



Problem21B.java

```
package com.jlcindia.arrays;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
// Problem 21B: Rotate the Elements Right side K times.
//LeetCode - 189
public class Problem21B {
    public static void reverse(int arr[],int start,int end) {
        while(start<end) {
            int temp = arr[start];
            arr[start] = arr[end];
            arr[end] = temp;
            start++;
            end--;
        }
    }
    public static void rotate(int[] arr, int k) { // k=3
        int n=arr.length;
        k=k%n;

        reverse(arr,0,n-1);
        reverse(arr,0,k-1);
        reverse(arr,k,n-1);
    }
    public static void main(String[] args) {
        int arr[] = { 1, 2, 3, 4, 5, 6,7 };
        System.out.println(Arrays.toString(arr));

        rotate(arr,3);
        System.out.println(Arrays.toString(arr));
    }
}
// Time Complexity - - O(3n) => O(n)
// Aux Space - O(1)
```



Problem22A.java

```
package com.jlcindia.arrays;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
//Problem 22: Maximum Sum of K Consecutive Elements
public class Problem22A {
    public static int maxSum(int[] arr, int k) {

        int n = arr.length;
        int result = Integer.MIN_VALUE;

        for (int i = 0; i + k - 1 < n; i++) {
            int curSum = 0;
            for (int j = 0; j < k; j++) {
                curSum = curSum + arr[i + j];
            }
            result = Math.max(result, curSum);
        }
        return result;
    }
    public static void main(String[] args) {
        //int arr[] = { 1, 8, 30, -5, 20, 7 };
        int arr[] = {10,5,-2,20,1};
        int k = 3;
        System.out.println(Arrays.toString(arr));

        int result = maxSum(arr, k);
        System.out.println(result);
    }
}

//Time Complexity - O(n * k)
//Aux Space - O(1)
```



Problem22B.java

```
package com.jlcindia.arrays;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
//Problem 22: Maximum Sum of K Consecutive Elements
// Sliding Window Technique
public class Problem22B {
    public static int maxSum(int[] arr, int k) {
        int n = arr.length;
        int curSum = 0;

        for (int i = 0; i < k; i++) {
            curSum = curSum + arr[i];
        }
        int result = curSum;
        for (int i = k; i < n; i++) {
            curSum = curSum + arr[i] - arr[i - k];
            result = Math.max(result, curSum);
        }
        return result;
    }
    public static void main(String[] args) {
        int arr[] = { 1, 8, 30, -5, 20, 7 };
        int k = 3;
        System.out.println(Arrays.toString(arr));

        int result = maxSum(arr, k);
        System.out.println(result);
    }
}

//Time Complexity - O(n)
//Aux Space - O(1)
```



Problem23.java

```
package com.jlcindia.arrays;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
//Problem 23: SubArray Sum equals to K
//Sliding Window Technique
public class Problem23 {
    public static boolean isSubSum(int[] arr, int sum) {
        int n = arr.length;
        int curSum = 0;
        int start=0;
        for (int end = 0; end < n;end++) {
            curSum = curSum + arr[end];
            while(sum<curSum) {
                curSum = curSum - arr[start];
                start++;
            }
            if(sum==curSum)
                return true;
        }
        return false;
    }
    public static void main(String[] args) {
        //int arr[] = { 1, 4, 20, 3, 10, 5 };           //int sum =33 ;
        int arr[] = {1,4,0,0,3, 10, 5};               int sum =7 ;
        System.out.println(Arrays.toString(arr));
        boolean result = isSubSum(arr, sum);
        System.out.println(result);
    }
}
//Time Complexity - O(n)
//Aux Space - O(1)
```



Problem24A.java

```
package com.jlcindia.arrays;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
//Problem 24: Maximum SubArray
//LeetCode - 53
public class Problem24A {
    public static int maxSubArray(int[] arr) {
        int n = arr.length;
        int result = arr[0];

        for (int i = 0; i < n; i++) {
            int curSum = 0;
            for (int j = i; j < n; j++) {
                curSum = curSum + arr[j];
                result = Math.max(result, curSum);
            }
        }
        return result;
    }

    public static void main(String[] args) {

        //int arr[] = {-2,1,-3,4,-1,2,1,-5,4};
        int arr[] = {5,4,-1,7,8};
        System.out.println(Arrays.toString(arr));

        int result = maxSubArray(arr);
        System.out.println(result);
    }
}
//Time Complexity - O(n ^2)
//Aux Space - O(1)
```



Problem24B.java

```
package com.jlcindia.arrays;
/*
 * @Author : Srinivas Dande
 * @Company: Java Learning Center
 */
//Problem 24: Maximum SubArray
//LeetCode - 53
public class Problem24B {
    public static int maxSubArray(int[] arr) {

        int n = arr.length;
        int result = arr[0];
        int maxLeft = arr[0];

        for (int i = 1; i < n; i++) {
            maxLeft = Math.max(maxLeft + arr[i], arr[i]);
            result = Math.max(result, maxLeft);
        }
        return result;
    }
    public static void main(String[] args) {

        int arr[] = {-2,1,-3,4,-1,2,1,-5,4};
        //int arr[] = { 5, 4, -1, 7, 8 };
        System.out.println(Arrays.toString(arr));

        int result = maxSubArray(arr);
        System.out.println(result);
    }
}

//Time Complexity - O(n)
//Aux Space - O(1)
```